

HDOC DAY-14 ACTIVITY

QUESTION 1 – Menu-Driven Digit Operations

Write a menu-driven C program using switch-case to input a positive integer and perform one of the following operations based on the user's choice:

- Check whether all its digits are unique.
 - E.g. 1: Number: 12345, Output: Unique
 - E.g. 2: Number: 12234, Output: Not Unique
- Find the frequency of each digit from `0` to `9`.
 - E.g. Number: 112333, freq_0 = 0, freq_1 = 2, freq_2 = 1, freq_3 = 3, freq_4 = 0, freq_5 = 0, freq_6 = 0, freq_7 = 0, freq_8 = 0, freq_9 = 0

1. Problem Understanding

- Input: A positive integer and the user's menu choice.
- Output: The result of the operation selected through switch-case.

Operation Summary

Choice	Operation	Main Logic
1	Check whether all digits are unique	Track the occurrence of each digit and check whether any digit occurs more than once.
2	Find frequency of each digit from 0 to 9	Extract each digit and increment its corresponding frequency counter.
Other	Invalid choice	Display an appropriate message.

2. Standard Test Cases

Test Case	Choice	Input	Expected Output	Actual Output	Result
1	1	n = 12345	Unique	Unique	Pass
2	1	n = 12234	Not Unique	Not Unique	Pass
3	2	n = 112333	freq_0 = 0; freq_1 = 2; freq_2 = 1; freq_3 = 3; freq_4-9 = 0	freq_0 = 0; freq_1 = 2; freq_2 = 1; freq_3 = 3; freq_4-9 = 0	Pass
4	2	n = 55555	freq_5 = 5;	freq_5 = 5;	Pass

			All other frequencies = 0	All other frequencies = 0	
--	--	--	------------------------------	------------------------------	--

3. Algorithm

A. Common Menu Logic

1. Start the program.
2. Declare the required variables.
3. Read the positive integer.
4. Display the menu and read the user's choice.
5. Use switch-case to perform the selected operation.
6. Display the result.
7. Stop the program.

B. Case 1 – Check Unique Digits

8. Initialize an array count[10] to zero.
9. Extract the last digit using $n \% 10$.
10. Increment the count of the extracted digit.
11. Remove the last digit using $n / 10$ and repeat until all digits are processed.
12. Check whether any digit has a count greater than 1.
13. If any digit occurs more than once, display Not Unique; otherwise display Unique.

C. Case 2 – Digit Frequency

14. Initialize an array freq[10] to zero.
15. Extract the last digit using $n \% 10$.
16. Increment freq[digit].
17. Remove the last digit using $n / 10$ and repeat until all digits are processed.
18. Display the frequency of each digit from 0 to 9.

5. C Program

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
int n, choice, digit, i, temp;
```

```
int count[10] = {0};
```

```
int freq[10] = {0};
```

```
int unique;
```

```
printf("Enter a positive integer: ");
```

```
scanf("%d", &n);

printf("\nMenu\n");

printf("1. Check whether all digits are unique\n");
printf("2. Find frequency of each digit from 0 to 9\n");

printf("Enter your choice: ");

scanf("%d", &choice);

switch (choice)

{

case 1:

temp = n;

unique = 1;

while (temp > 0)

{

digit = temp % 10;

count[digit]++;

if (count[digit] > 1)

unique = 0;

temp = temp / 10;

}

if (unique)

printf("Output: Unique\n");

else

printf("Output: Not Unique\n");

break;

case 2:

temp = n;

while (temp > 0)
```

```

{

digit = temp % 10;

freq[digit]++;

temp = temp / 10;

}

for (i = 0; i < 10; i++)

printf("freq_ %d = %d\n", i, freq[i]);

break;

default:

printf("Invalid choice.\n");

}

return 0;

}

```

6. Compilation and Execution Evidence

```

kali@kali: ~/Desktop
Session Actions Edit View Help

(kali@kali)-[~/Desktop]
$ gedit question.c

** (gedit:2215): WARNING **: 09:17:41.422: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
** (gedit:2215): WARNING **: 09:17:41.422: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found

(kali@kali)-[~/Desktop]
$ gcc question.c -o question

(kali@kali)-[~/Desktop]
$ ./question
Enter a positive integer: 12345

Menu
1. Check whether all digits are unique
2. Find frequency of each digit from 0 to 9
Enter your choice: 1
Output: Unique

(kali@kali)-[~/Desktop]
$ ./question
Enter a positive integer: 12234

Menu
1. Check whether all digits are unique
2. Find frequency of each digit from 0 to 9
Enter your choice: 1
Output: Not Unique

(kali@kali)-[~/Desktop]
$ ./question
Enter a positive integer: 112333

Menu
1. Check whether all digits are unique
2. Find frequency of each digit from 0 to 9
Enter your choice: 2
freq_0 = 0
freq_1 = 2

```

```
Session Actions Edit View Help
(kali@kali)-[~/Desktop]
└─$ ./question
Enter a positive integer: 112333

Menu
1. Check whether all digits are unique
2. Find frequency of each digit from 0 to 9
Enter your choice: 2
freq_0 = 0
freq_1 = 2
freq_2 = 1
freq_3 = 3
freq_4 = 0
freq_5 = 0
freq_6 = 0
freq_7 = 0
freq_8 = 0
freq_9 = 0

(kali@kali)-[~/Desktop]
└─$ ./question
Enter a positive integer: 55555

Menu
1. Check whether all digits are unique
2. Find frequency of each digit from 0 to 9
Enter your choice: 2
freq_0 = 0
freq_1 = 0
freq_2 = 0
freq_3 = 0
freq_4 = 0
freq_5 = 5
freq_6 = 0
freq_7 = 0
freq_8 = 0
freq_9 = 0

(kali@kali)-[~/Desktop]
└─$
```

7. Verification of All Test Cases

Test Case	Choice	Input	Expected Output	Actual Output	Result
1	1	n = 12345	Unique	Unique	Pass
2	1	n = 12234	Not Unique	Not Unique	Pass
3	2	n = 112333	freq_0 = 0; freq_1 = 2; freq_2 = 1; freq_3 = 3; freq_4–9 = 0	freq_0 = 0; freq_1 = 2; freq_2 = 1; freq_3 = 3; freq_4–9 = 0	Pass
4	2	n = 55555	freq_5 = 5; All other frequencies = 0	freq_5 = 5; All other frequencies = 0	Pass